Cross Entropy Loss (Log Loss)

- Intuition about cross entropy
- Derivation of cross entropy loss
- Why is it used in machine learning

Cross Entropy Loss

$$H(P^* \mid P) = -\sum_i P^*(i) \log P(i)$$

true

predicted

- Dominant loss function used in machine learning classification tasks.
- $p^*(i)$: true probability $P(i)$: predicted probability
- i : class number



Basic image classification:

Given the image of an animal we would like to classify which animal it is.



Dog
Cat
Fox
Elephant
Lion
Mouse
Bird
Pokémon
Fish
None

} classes



MODEL

Parameters: $\boldsymbol{\theta}$

DOG

Dog
Cat
Fox
Elephant
Lion
Mouse
Bird
Pokémon
Fish
None



MODEL

Parameters: $\boldsymbol{\theta}$

0

Dog	0
Cat	1
Fox	2
Elephant	3
Lion	4
Mouse	5
Bird	6
Pokémon	7
Fish	8
None	9

- How do we represent the output of the model?
- The model could output a number which indicates the given class.
- Difficult to calculate the "distance" between classes.



MODEL
Parameters: $\boldsymbol{\theta}$

0

	probabilities
Dog	0.99
Cat	0.001
Fox	0.001
Elephant	0.001
Lion	0.001
Mouse	0.001
Bird	0.001
Pokémon	0.001
Fish	0.001
None	0.001

- Using a probability distribution as output is more natural.
  - 99% dog
  - 1% other animal

  it gives the correct class with a level of "certainty".



MODEL
Parameters: $\boldsymbol{\theta}$

0

	probabilities
Dog	1.00
Cat	0.00
Fox	0.00
Elephant	0.00
Lion	0.00
Mouse	0.00
Bird	0.00
Pokémon	0.00
Fish	0.00
None	0.00

- If max. probability gives a sufficiently "certain" level, we can assign the probability of 1 to that class.



	probabilities
Dog	1.00
Cat	0.00
Fox	0.00
Elephant	0.00
Lion	0.00
Mouse	0.00
Bird	0.00
Pokémon	0.00
Fish	0.00
None	0.00



	probabilities
Dog	0.50
Cat	0.50
Fox	0.00
Elephant	0.00
Lion	0.00
Mouse	0.00
Bird	0.00
Pokémon	0.00
Fish	0.00
None	0.00

Given an ambiguous image, representing the output as a probability distribution helps us convey maximum information about the image.

output is a probability distribution



MODEL

Parameters: $\boldsymbol{\theta}$

$\mathbf{P(Dog}|x_i)$
$\mathbf{P(Cat}|x_i)$
$\mathbf{P(Fox}|x_i)$
$\mathbf{P(Elephant}|x_i)$
$\mathbf{P(Lion}|x_i)$
$\mathbf{P(Mouse}|x_i)$
$\mathbf{P(Bird}|x_i)$
$\mathbf{P(Pokémon}|x_i)$
$\mathbf{P(Fish}|x_i)$
$\mathbf{P(None}|x_i)$

INPUT IMAGE

x_i

PREDICTED CLASS DISTIRBUTION

$\mathbf{P}(y|x_i;\boldsymbol{\theta})$



MODEL

Parameters: $\boldsymbol{\theta}$

	predicted	target
P(Dog$\|x_i$)		1
P(Cat $\|x_i$)		0
P(Fox$\|x_i$)		0
P(Elephant$\|x_i$)		0
P(Lion$\|x_i$)		0
P(Mouse$\|x_i$)		0
P(Bird$\|x_i$)		0
P(Pokémon$\|x_i$)		0
P(Fish$\|x_i$)		0
P(None$\|x_i$)		0

INPUT IMAGE x_i

PREDICTED CLASS DISTIRBUTION $\mathbf{P}(\boldsymbol{y}|\boldsymbol{x_i};\boldsymbol{\theta})$

TRUE CLASS DISTIRBUTION $\boldsymbol{P^*(y|x_i)}$

- If we know the true class distribution, we can adjust the model parameters θ, so that predicted and true class distributions are in agreement.

- What is the best "loss function" to ensure that the predicted class distribution is close to the true class distribution?
- KL-divergence is a natural way of measuring the distance between probability distributions.

$$D_{KL}(P||Q) = \sum_i P(i)\log\frac{P(i)}{Q(i)}$$

INPUT IMAGE	PREDICTED CLASS DISTIRBUTION	TRUE CLASS DISTIRBUTION
$\boldsymbol{x_i}$	$\mathbf{P}(\boldsymbol{y}|\boldsymbol{x_i};\boldsymbol{\theta})$	$\boldsymbol{P^*(y|x_i)}$

- Intuitive loss formula: minimize the KL-divergence between the model and the true distribution.

TRUE CLASS DISTIRBUTION

PREDICTED CLASS DISTIRBUTION

$$D_{KL}\left(\mathbf{P}^*(\boldsymbol{y}|\boldsymbol{x_i})||\boldsymbol{P}(\boldsymbol{y}|\boldsymbol{x_i};\boldsymbol{\theta})\right)$$

$$D_{KL}(P||Q) = \sum_i P(i)\log\frac{P(i)}{Q(i)}$$

INPUT IMAGE

$\boldsymbol{x_i}$

PREDICTED CLASS DISTIRBUTION

$\mathbf{P}(\boldsymbol{y}|\boldsymbol{x_i};\boldsymbol{\theta})$

TRUE CLASS DISTIRBUTION

$\boldsymbol{P}^*(\boldsymbol{y}|\boldsymbol{x_i})$

TRUE CLASS DISTIRBUTION

PREDICTED CLASS DISTIRBUTION

$$D_{KL}\left(\mathbf{P}^*(\boldsymbol{y}|\boldsymbol{x_i})||\boldsymbol{P}(\boldsymbol{y}|\boldsymbol{x_i};\boldsymbol{\theta})\right)$$

$$D_{KL}(P||Q) = \sum_i P(i)\log\frac{P(i)}{Q(i)}$$

$$D_{KL}(P^*||P) = \sum_y P^*(y|x_i)\log\frac{P^*(y|x_i)}{P(y|x_i;\theta)}$$

modify the fomula to our notation.

$$D_{KL}(P^*||P) = \sum_y P^*(y|x_i) \log \frac{P^*(y|x_i)}{P(y|x_i;\theta)}$$

$$= \sum_y P^*(y|x_i) \left[\log P^*(y|x_i) \ - \log P(y|x_i;\theta)\right]$$

$$D_{KL}(P^*||P) = \sum_y P^*(y|x_i) \log \frac{P^*(y|x_i)}{P(y|x_i;\theta)}$$

$$= \sum_y P^*(y|x_i) \left[\log P^*(y|x_i) \; - \log P(y|x_i;\theta)\right]$$

DOESN'T DEPEND ON θ

$$= \sum_y \overbrace{P^*(y|x_i) \log P^*(y|x_i)} - \sum_y P^*(y|x_i) \log P(y|x_i;\theta)$$

$$D_{KL}(P^*||P) = \sum_y P^*(y|x_i) \log \frac{P^*(y|x_i)}{P(y|x_i;\theta)}$$

simplify using log rules.

$$= \sum_y P^*(y|x_i) \left[\log P^*(y|x_i) - \log P(y|x_i;\theta)\right]$$

DOESN'T DEPEND ON θ

$$= \sum_y \overline{P^*(y|x_i) \log P^*(y|x_i)} - \sum_y P^*(y|x_i) \log P(y|x_i;\theta)$$

$$\underset{\theta}{\operatorname{argmin}}\, D_{KL}(P^*||P)$$

$$D_{KL}(P^*||P) = \sum_y P^*(y|x_i) \log \frac{P^*(y|x_i)}{P(y|x_i;\theta)}$$

$$= \sum_y P^*(y|x_i) \left[\log P^*(y|x_i) - \log P(y|x_i;\theta)\right]$$

DOESN'T DEPEND ON θ

$$= \sum_y \overbrace{P^*(y|x_i) \log P^*(y|x_i)} - \sum_y P^*(y|x_i) \log P(y|x_i;\theta)$$

$$\underset{\theta}{\operatorname{argmin}}\, D_{KL}(P^*||P) \equiv \underset{\theta}{\operatorname{argmin}} \underbrace{-\sum_y P^*(y|x_i) \log P(y|x_i;\theta)}$$

$H(p^*, p)$: cross-entropy loss



MODEL

Parameters: $\boldsymbol{\theta}$

P(Dog$|x_i$)
P(Cat $|x_i$)
P(Fox$|x_i$)
P(Elephant$|x_i$)
P(Lion$|x_i$)
P(Mouse$|x_i$)
P(Bird$|x_i$)
P(Pokémon$|x_i$)
P(Fish$|x_i$)
P(None$|x_i$)

$\mathbf{P}(\boldsymbol{y}|\boldsymbol{x_i};\boldsymbol{\theta})$

TRUE CLASS DISTIRBUTION

$\boldsymbol{P^*(y|x_i)}$

$$\underset{\theta}{\operatorname{argmin}}\, D_{KL}(P^*||P) \equiv \underset{\theta}{\operatorname{argmin}}\, H(P^*, P)$$

KL loss

cross-entropy loss

- We need to make sure that the output is a valid probability distribution: values are between 0 and 1; they add up to 1.



MODEL
Parameters: $\boldsymbol{\theta}$

logits: z

$P(\text{Dog}\mid x_i)$	4.4	Invalid as probabilities can't be greater than 1
$P(\text{Cat}\mid x_i)$	0.2	
$P(\text{Fox}\mid x_i)$	0.2	
$P(\text{Elephant}\mid x_i)$	0.1	
$P(\text{Lion}\mid x_i)$	0.0	
$P(\text{Mouse}\mid x_i)$	-0.4	Invalid as probabilities can't be negative
$P(\text{Bird}\mid x_i)$	0.2	
$P(\text{Pokémon}\mid x_i)$	0.3	
$P(\text{Fish}\mid x_i)$	0.0	
$P(\text{None}\mid x_i)$	0.2	

Invalid as distribution must sum to 1

- By using a softmax unit at the end, the scores can be normalized into a probability distribution, where all outputs are between 0 and 1, and they sum up to 1.
- Softmax function:

$$\sigma(z)_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}, \quad i = 1, \dots, K \quad (K \rightarrow \#\text{classes})$$

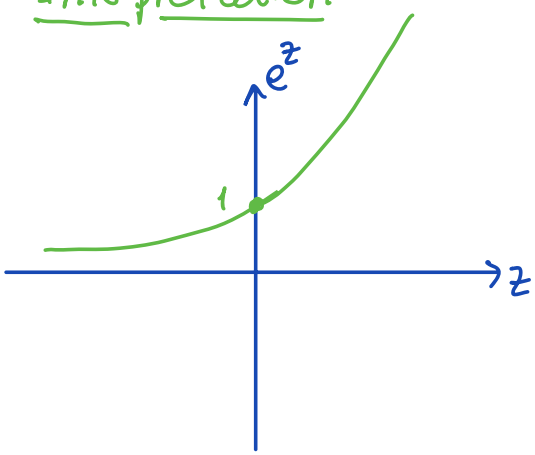
$$z = (z_1, \dots, z_K) \in \mathbb{R}^K$$

$$z = \begin{bmatrix} 4.4 \\ 0.2 \\ 0.2 \\ 0.1 \\ 0.0 \\ -0.4 \\ 0.2 \\ 0.3 \\ 0.0 \\ 0.2 \end{bmatrix}$$

- Softmax function applies the standard exponential function to each element z_i of the input vector z, and normalizes these values by dividing by the sum of all these exponentials.
- This normalization ensures that the sum of the components of the output vector $\sigma(z)$ is 1.

- The term "softmax" derives from the amplifying effects of the exponential on any maxima in the input vector.
- For example, the standard softmax of $(1, 2, 8)$ is approximately $(0.001, 0.002, 0.997)$, which amounts to assigning almost all of the total unit weight in the result to the position of the vector's maximal element (of 8).

- Interpretation:



- Exponential function maps the negative numbers to: $[0, 1)$

- The name "softmax" is misleading; the function is not a "smooth maximum" (a "smooth approximation" to the "maximum" function, but rather a smooth approximation to the "arg max" function: the function whose value is which index has the maximum.

Note: Consider the "arg max" function with one-hot representation of the output (assuming there is a unique maximum arg:

$\arg\max(z_1, \ldots, z_n) = (y_1, \ldots, y_n) = (0, \ldots, 0, 1, 0, \ldots, 0)$

where the coordinate $y_i = 1$ if and only if i is the arg max of $(z_1, \ldots, z_n)$; meaning z_i is the unique maximum value of $(z_1, \ldots, z_n)$.

For example, $\arg\max(1, 5, 10) = (0, 0, 1)$

since the third argument is the maximum.



MODEL

Parameters: $\boldsymbol{\theta}$

	After softmax
P(Dog$\|x_i$)	0.80
P(Cat $\|x_i$)	0.03
P(Fox$\|x_i$)	0.03
P(Elephant$\|x_i$)	0.02
P(Lion$\|x_i$)	0.01
P(Mouse$\|x_i$)	0.00
P(Bird$\|x_i$)	0.03
P(Pokémon$\|x_i$)	0.04
P(Fish$\|x_i$)	0.01
P(None$\|x_i$)	0.03

- A softmax unit is almost always used at the end of deep learning based classification systems: they convert logits to probabilities.



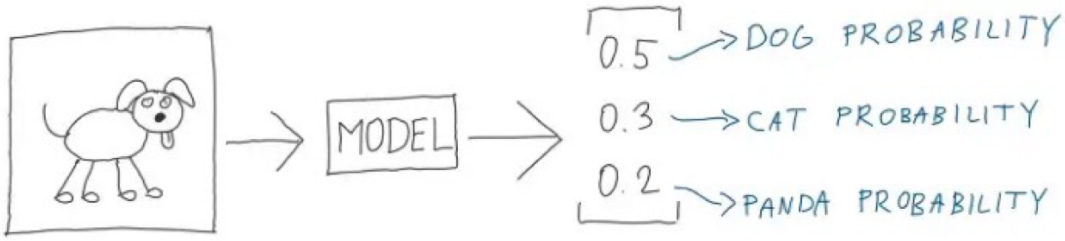
MODEL

Parameters: $\boldsymbol{\theta}$

	After softmax	LABELS
P(Dog$\|x_i$)	0.80	1.00
P(Cat $\|x_i$)	0.03	0.00
P(Fox$\|x_i$)	0.03	0.00
P(Elephant$\|x_i$)	0.02	0.00
P(Lion$\|x_i$)	0.01	0.00
P(Mouse$\|x_i$)	0.00	0.00
P(Bird$\|x_i$)	0.03	0.00
P(Pokémon$\|x_i$)	0.04	0.00
P(Fish$\|x_i$)	0.01	0.00
P(None$\|x_i$)	0.03	0.00

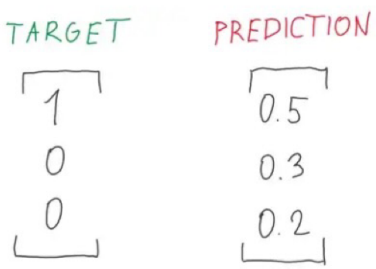
- The category labels can be converted to hard probability distributions using one-hot encoding.

* Let's focus on the following simplified case:



Multi-class classification

- Let's compute the cross-entropy loss for this image.
- *Loss is a measure of performance of a model. The lower, the better. When learning, the model aims to get the lowest loss possible.*
- The target represents probabilities for all classes — dog, cat, and panda.
- *The target for multi-class classification is a one-hot vector, meaning it has 1 on a single position and 0's everywhere else.*



Cross-entropy loss:

$$H(P^* \mid P) = -\sum_i P^*(i) \log P(i)$$

(i: class index; $P^*(i)$: true; $P(i)$: predicted)

e.g: 3 classes:

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad \begin{bmatrix} 0.5 \\ 0.3 \\ 0.2 \end{bmatrix} \begin{matrix} \rightarrow \text{dog} \\ \rightarrow \text{cat} \\ \rightarrow \text{panda} \end{matrix}$$

$$= -(\underbrace{1 \cdot \log 0.5}_{\text{loss for dog class}} + \underbrace{0 \cdot \log 0.3}_{\text{loss for cat class}} + \underbrace{0 \cdot \log 0.2}_{\text{loss for panda class}})$$

$$= -\log 0.5$$

- Why sum up over all classes if the loss for most of them are zero? We can forget targets and predictions for all other classes and compute only the loss for the "hot" class.

Hence, cross-entropy loss can be simplified as follows:

$$H(P^*, P) = -\log P(x)$$

(x is the class that is 1 in our target.)

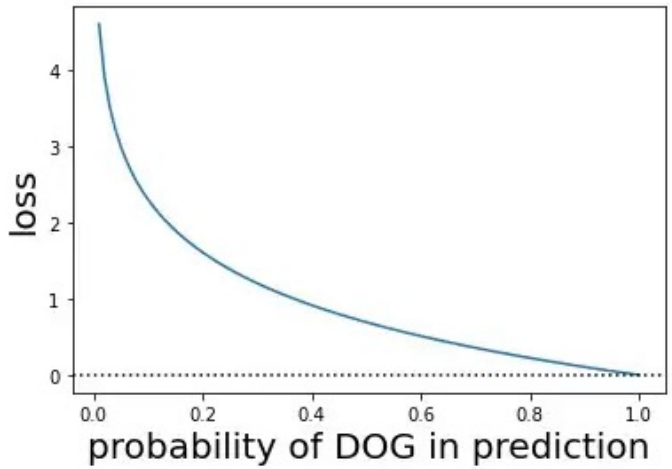
This is called categorical cross-entropy loss, a special case where our target is a one-hot vector.

Multi-class classification
Cross-entropy Loss

$$\text{Loss for DOG} = -p(\text{DOG}) \cdot \log q(\text{DOG})$$

$$= - \ 1 \ \cdot \log 0.5$$

$$= 0.693...$$



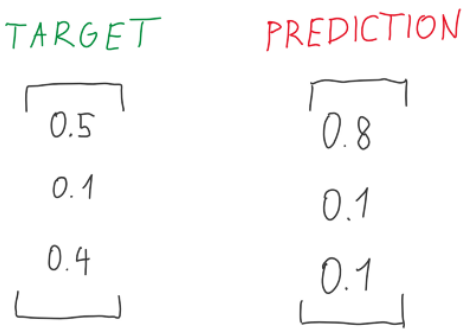
- The loss is 0 when the prediction is 1 (the same as the target).
- The loss is infinity if the prediction is 0 (the complete opposite of our target).
- What if we predict something in the middle?
- The loss gets steeper, the further away from the target we get.

Multi-class classification
Cross-entropy Loss

- **Why is the loss 0 for the cat and panda classes?**
- It looks like we are rewarding the model with low loss, even if it predicts a high probability for a class that is not present in the image.
- We don’t mind if the model predicts that there is a cat with an 80% probability if there is none because then it has only 20% left to predict the correct class. There, the loss will be that much bigger.
- In other words — we don’t care on which classes the model wastes the predicted probabilities, only how correctly it identifies the only present class.

Multi-class classification

- The cross-entropy loss works even for distributions that are not one-hot vectors. The loss would work even for this task:



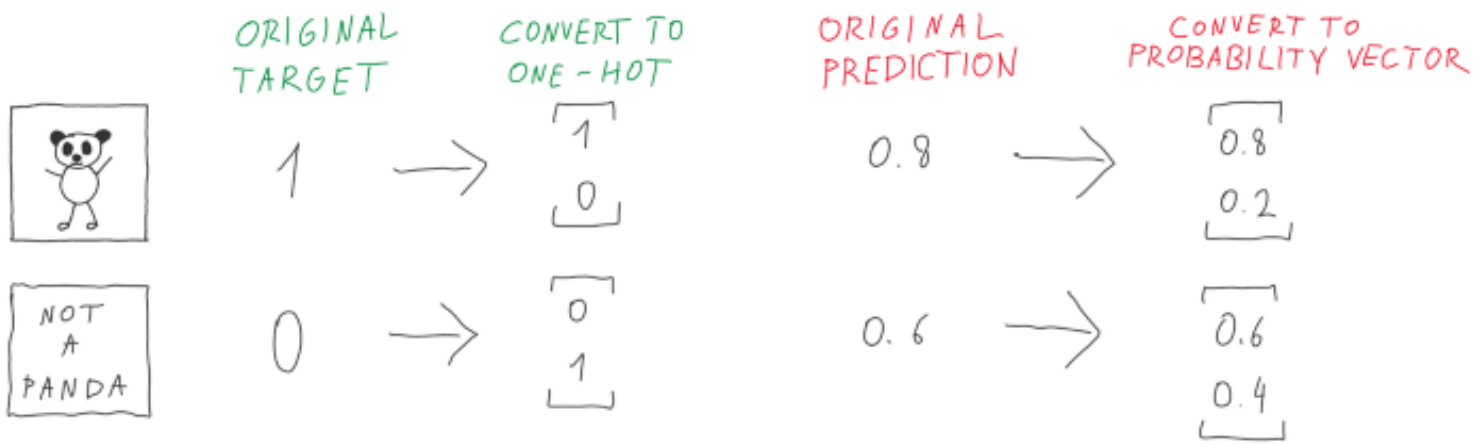
- With the cross-entropy, we would still be able to compute the loss, and it would be minimal if all the classes would be correct, and still have the property of punishing bigger mistakes much more.
- In our one-hot target example, the entropy was conveniently 0, so the minimal loss was 0. If your target is a probability vector that is **not** one-hot, entropy (minimal loss) will be bigger than 0, but you can still use the cross-entropy loss just fine.

* The cross-entropy loss discussed above is the loss for 1 image. (the image of a dog).
* If we wanted the loss for our batch or the whole dataset, we would just sum up the losses of the individual images.

Binary Classification

Binary Classification

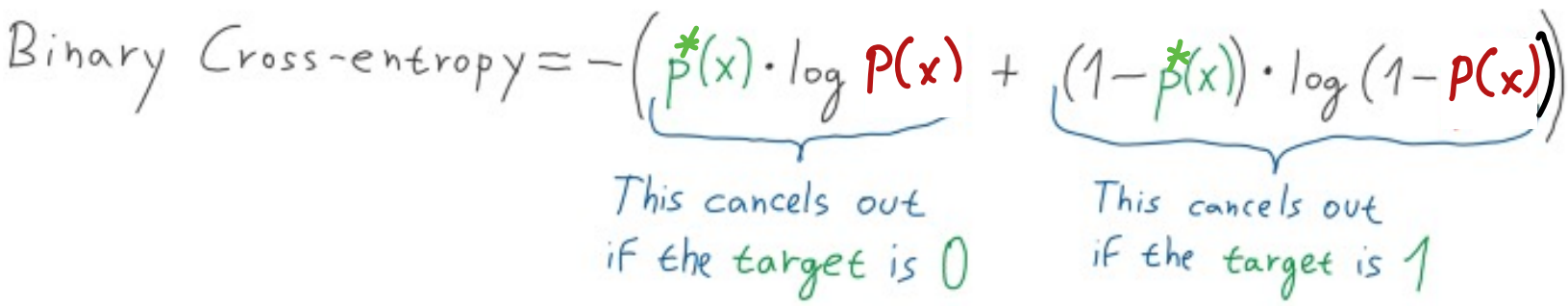
- **Binary cross-entropy** is another special case of cross-entropy — used if our target is either 0 or 1. In a neural network, you typically achieve this prediction by sigmoid activation.
- The target is **not** a probability vector. We can still use cross-entropy with a little trick.
- We want to predict whether the image contains a panda or not.



Binary Classification:

Binary cross-entropy

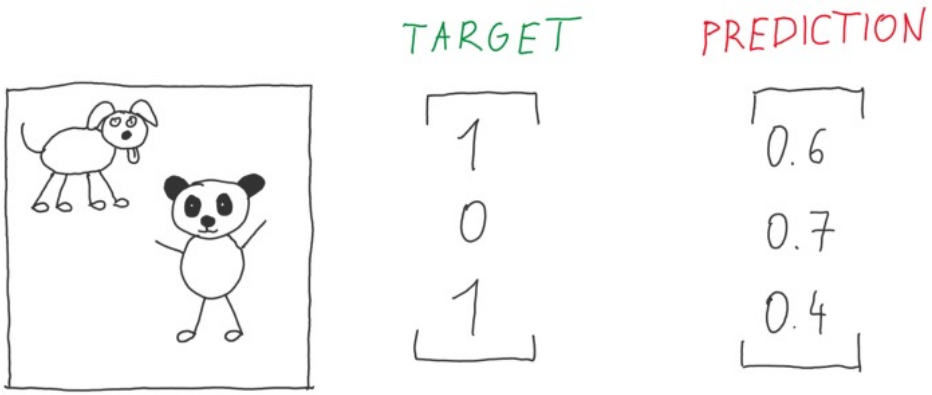
- This is the same as if we would convert the target to a one-hot vector and our prediction to a probability vector — the probability of panda would be the same as the prediction and probability of not-a-panda would be 1-prediction. In other words, if we predict 0.6, that means we are saying that it's 60% a panda and 40% not-a-panda.
- This loss can be computed with the cross-entropy function since we are now comparing just two probability vectors or even with categorical cross-entropy since our target is a one-hot vector.
- It can also be computed without the conversion with a **binary cross-entropy**:



Multi-label classification

Multi-label classification

- Cross-entropy can also be used as a loss function for a multi-label problem with this simple trick:



- Notice our target and prediction are **not** a probability vector. It's possible that there are all classes in the image, as well as none of them. In a neural network, you typically achieve this by sigmoid activation.

Multi-label classification

- We can look at this problem as multiple binary classification subtasks. Let's say we want to only predict if there is a dog or not.
- We know that our target is 1 and we have predicted 0.6.
- We will compute the binary cross-entropy for this subtask:

$$\text{Binary Cross-entropy}_{\text{DOG}} = -\left(p^*(x) \cdot \log p(x) + (1-p^*(x)) \cdot \log(1-p(x))\right)$$

$$= -\left(1 \cdot \log 0.6 + (1-1) \cdot \log(1-0.6)\right)$$

$$= -\left(\log 0.6 + 0\right)$$

$$= 0.510\ldots$$

Multi-label classification

- And do the same for the other classes.
- For a cat, our target is 0, so the other part of binary cross-entropy cancels out:

$$\text{Binary Cross-entropy}_{CAT} = -\left(p^*(x) \cdot \log p(x) + (1 - p^*(x)) \cdot \log (1 - p(x)) \right)$$

$$= -\left(0 \cdot \log 0.7 + (1-0) \cdot \log (1 - 0.7) \right)$$

$$= -\left(0 + \log 0.3 \right)$$

$$= 1.20\ldots$$

Multi-label classification

$$\text{Binary Cross-entropy}_{\text{PANDA}} = -\left(p^*(x) \cdot \log P(x) + (1 - p^*(x)) \cdot \log (1 - P(x)) \right)$$

$$= -\left(1 \cdot \log 0.4 + (1 - 1) \cdot \log (1 - 0.4) \right)$$

$$= -\left(\log 0.4 + 0 \right)$$

$$= 0.916\ldots$$

- And sum up the losses for each subtask:

$$\text{Total Loss} = \text{Binary Cross-entropy}_{\text{DOG}} + \text{Binary Cross-entropy}_{\text{CAT}} + \text{Binary Cross-entropy}_{\text{PANDA}}$$

$$= 2.631\ldots$$

Summary

Cross-entropy — the general formula, used for calculating loss among two probability vectors. The more we are away from our target, the more the error grows — similar idea to square error.

Cross-entropy $= H(P^* | P) = -\sum_i P^*(i) \log P(i)$

Summary

Multi-class classification — we use multi-class cross-entropy — a specific case of cross-entropy where the target is a one-hot encoded vector. It can be computed with the cross-entropy formula but can be simplified.

Categorical Cross-entropy = $-\log P(x)$

x is the class that is 1 in our TARGET

Summary

Binary classification — we use binary cross-entropy — a specific case of cross-entropy where our target is 0 or 1. It can be computed with the cross-entropy formula if we convert the target to a one-hot vector like [0,1] or [1,0] and the predictions respectively. We can compute it even without this conversion, with the simplified formula.

$$\text{Binary Cross-entropy} = -\Big(\underbrace{p^*(x)\cdot\log p(x)}_{\text{This cancels out if the target is 0}} + \underbrace{(1-p^*(x))\cdot\log(1-p(x))}_{\text{This cancels out if the target is 1}}\Big)$$

Summary

Multi-label classification — Our target can represent multiple (or even zero) classes at once. We compute the binary cross-entropy for each class separately and then sum them up for the complete loss.

$$\text{Total Loss} = \sum_{x} \text{Binary Cross-entropy}_x$$

Reference

- "Cross-Enropy for classification", Vlastimil Martinek
- https://towardsdatascience.com/cross-entropy-for-classification-d98e7f974451

"Intuitively Understanding the Cross Entropy Loss", Adian Liusie